universität innsbruck
Department of
Computer Science

Recommender Systems

Content-Based Filtering

Eva Zangerle

Recap: Recommender System Paradigms

- Two major paradigms:
  - Content-based filtering (CBF)
    - Recommending items based on their content, described by features.
    - Example: similar movies based on genre, actors, director, etc.
  - Collaborative filtering (CF)
    - Recommending items based on user-item interactions (ratings).
    - Example: users who bought this item also bought that item.
  - Additionally, hybrid approaches combine both paradigms to leverage their strengths and mitigate their weaknesses.

Motivation

Content-based Filtering

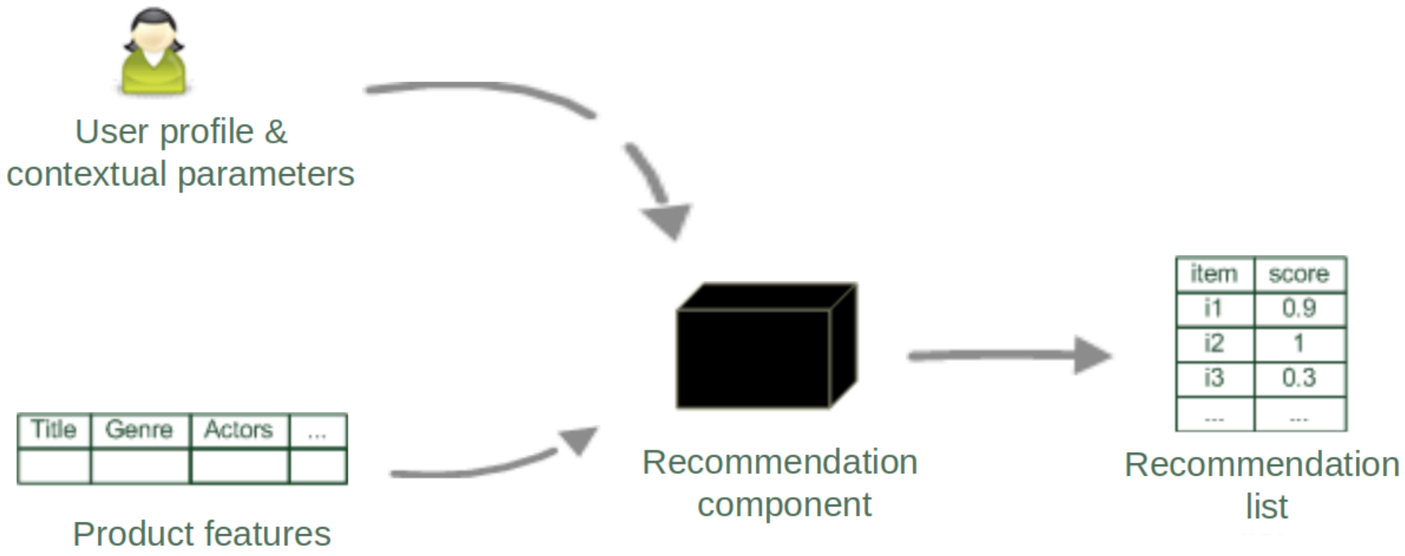


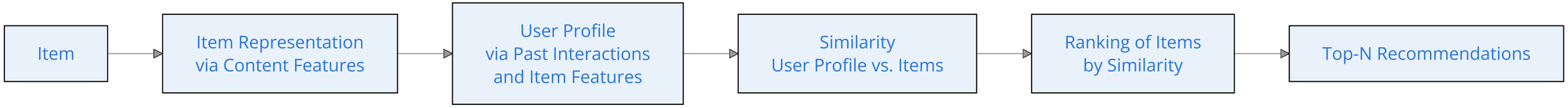
Figure adapted from Dietmar Jannach, Markus Zanker, Alexander Felfernig, and Gerhard Friedrich. 2010. Recommender Systems: An Introduction (1st. ed.). Cambridge University Press, USA.

Content-Based Filtering: Idea

- Recommend items based on their content (item features).
- “If you liked this item, you will like similar items”.
- Instead of using (explicit) ratings, use computational features derived from item content to compute similarity ➜ item similarity via item features.

- Example: A user liked a sci-fi movie about space exploration.
  - Item features:
    - Genre: sci-fi
    - Keywords: space, exploration
    - Actors: Tom Hanks, Matt Damon
  - Recommendation: Movies with similar genre, theme, actors, director, style, etc.

Recommendation Process and Overview



- Step 1: Compute vector representation of items based on their content (features).
- Step 2: Build user profile from past interactions and item features of interacted items.
- Step 3: Compute similarity between user profile and items.
- Step 4: Rank items and recommend top-N items to the user.

Content

- Early CBF-recommendation techniques were applied to text documents.
- Nowadays, content is any information that can be used to describe an item:
  - Text (e.g., product descriptions, reviews, metadata)
  - Audio (e.g., acoustic features of music, speech features of podcasts)
  - Image (visual features, e.g., color, shape, texture)
  - Video (multi-modal features, e.g., visual, audio, text)
  - Metadata (e.g., genre, user-created tags, categories)
- Content is high-dimensional and heterogeneous, and can be represented in various ways (e.g., vector space model, latent semantic indexing, word embeddings, etc.).

Structured vs. Unstructured Content

- Structured content: Fixed classes/attributes: genre, year, artist, user-created tags, etc.
  ➔ low-dimensional, clear semantics, easy to interpret
- Unstructured content: Free text descriptions, images, audio, video, etc.
  ➔ rich in information, but high-dimensional and difficult to process
- Modern recommender systems often combine both types of content for better performance (e.g., metadata combined with text features).

Data Sources

- Product details (e.g., movie metadata, book descriptions, music attributes, images)
- Product reviews (e.g., Amazon, Epinions.com)
- Reviews and comments on social media (e.g., Youtube, Instagram)
- Postings in social media (e.g., microblogs, status messages)
- User tags (e.g., tagging-enabled platforms such as Flickr, Last.fm)
- Web pages devoted to the item (e.g., biography of artist, movie fanpage)
- Discussion forums
- Blogs (individual opinions, reviews, etc.)
- Wikis (collaborative knowledge about items, e.g., Wikipedia)

Item Representation

- Goal: represent items so that their content is captured and similarities can be computed efficiently.
- Items are mapped to vectors in a feature space:

$$x_i \in \mathbb{R}^d \ \forall i \in \mathcal{I}$$

  where x_i is the feature vector for item i, and d is the dimensionality of the feature space, and $\mathcal{I}$ is the set of items.
- Similarity is computed in this feature space (e.g., cosine similarity).
- Choice of features and representation is crucial for CBF performance
  - Poor representation ➜ poor recommendations
  - Good representation ➜ better recommendations

Text Content

Text Content

- Items are often represented by textual content (e.g., descriptions, reviews).
- Goal is to transform unstructured text describing the item into a numerical vector that captures the content of the item.
- Different approaches for representing textual content:
  - Bag-of-Words (BoW): vector of word counts or binary indicators for presence/absence of words (also keywords).
  - TF-IDF: term frequency-inverse document frequency weighting of words.

Text Content

- Different approaches for representing textual content (cont.):
  - Latent Semantic Indexing (LSI): dimensionality reduction technique that captures latent topics.
  - Word embeddings: dense vector representations of words (e.g., Word2Vec, GloVe, FastText).
  - Sentence/Document embeddings: dense vector representations of sentences or documents (e.g., Sent2Vec, Doc2Vec, BERT).
  - Transformer-based embeddings: contextualized embeddings from transformer models (e.g., BERT, RoBERTa, etc.) that capture context and semantics.

Keyword-Based Representation

Title	Genre	Author	Type	Price	Keywords
The Night of the Gun	Memoir	David Carr	Paperback	29.90	journalism, drug addiction, personal memoirs, New York
The Lace Reader	Fiction, Mystery	Brunonia Barry	Hardcover	49.90	American contemporary fiction, detective, historical, murder
Into the Fire	Romance, Suspense	Suzanne Brockmann	Hardcover	45.90	American fiction, murder, neo-Nazism,

Example taken from Dietmar Jannach, Markus Zanker, Alexander Felfernig, and Gerhard Friedrich. 2010. Recommender Systems: An Introduction (1st. ed.). Cambridge University Press, USA.

Keyword-Based Representation cont.

- Simple Approach: Items and users are represented as sets of keywords/words (often referred to as Bag-of-Words approach).
- Compute similarity of an unseen item with the user profile based on word overlap (e.g., using Dice or Jaccard coefficient) .
- Jaccard coefficient for two sets of keywords X and Y:

$$J(X, Y) = \frac{|X \cap Y|}{|X \cup Y|}$$

- Dice coefficient for two sets of keywords X and Y:

$$D(X, Y) = \frac{2|X \cap Y|}{|X| + |Y|}$$

Keyword-Based Representation cont.

We represent items and users as sets of keywords:

- The Night of the Gun: `{journalism, drug addiction, personal memoirs, new york}`
- The Lace Reader: `{american contemporary fiction, detective, historical, murder}`
- Into the Fire: `{american fiction, murder, neo-nazism}`

We represent users as set of keywords describing books they liked:

- User: `{detective, murder, new york}`

Keyword-Based Representation cont.

Recommendation based on keyword overlap

- Night of the Gun: `{new york}`
- Lace Reader: `{detective, murder}`
- Into the Fire: `{murder}`

Similarities

- Night of the Gun: $J = \frac{1}{6} \approx 0.16$, $D = \frac{2}{7} = 0.28$
- Lace Reader: $J = \frac{2}{5} = 0.40$, $D = \frac{4}{6} \approx 0.67$
- Into the Fire: $J = \frac{1}{5} = 0.20$, $D = \frac{2}{6} \approx 0.33$

Jaccard: $J(A, B) = \frac{|A \cap B|}{|A \cup B|}$

Dice: $D(A, B) = \frac{2|A \cap B|}{|A| + |B|}$

Keyword-Based Representation cont.

- Simple keyword representation has its problems:
  - All keywords are treated equally.
  - Not every word has similar importance.
  - Frequent words appear for many items, but are not informative.
- Lack of semantics: Synonyms not captured (e.g., “murder” vs. “homicide”).
- Documents with more keywords have a higher chance to have an overlap with the user profile as more keywords can be extracted
- Note that keywords can also be curated manually (annotation), which makes this approach viable also for non-textual items

Term Weighting

- Goal: Assign different weights to keywords based on their importance for the item.
- Idea:
  - Terms that are more frequent in a document are more important for that document.
  - Term that are frequent across all documents are less important for a specific document.

Term Weighting: Example

> One morning, when Gregor Samsa woke from troubled dreams, he found himself transformed in his bed into a horrible vermin. He lay on his armour-like back, and if he lifted his head a little he could see his brown belly, slightly domed and divided by arches into stiff sections. The bedding was hardly able to cover it and seemed ready to slide off any moment. His many legs, pitifully thin compared with the size of the rest of him, waved about helplessly as he looked.

Metamorphosis – Franz Kafka (https://www.gutenberg.org/files/5200/5200-0.txt)

Term Weighting: Term Frequency (TF)

Term frequency measures importance of a term t within a document d.

$$TF(t, d) = \sum_{i=1}^{|d|} \mathbf{1}(w_i = t)$$

- The higher the TF, the more relevant the term is for that item.
- Captures local importance of a term for a document.
- Does not consider how common the term is across all documents (i.e., does not distinguish informative vs common terms).

Term Weighting: Inverse Document Frequency (IDF)

Measures how rare a term is across the corpus

$$IDF(t) = \log \frac{N}{df(t)}$$

where N is the number of documents and $df(t)$ is the number of documents containing term t.

- Rare terms have a high IDF, common terms have a low IDF.
- High IDF means term is more informative, low IDF means term is less useful.
- Captures global importance of a term across the corpus.

Term Weighting: Monotonicity Assumptions

1. Rare terms are no less important than frequent ones (IDF assumption): Importance of a term for a document is the higher, the more rarely it appears across all documents (in the corpus).
2. Multiple appearances of a term in a document are no less important than single appearances (TF assumption): Importance of a term for a document is the higher, the more often it appears in the document.
3. Long documents are no more important than short documents (normalization assumption): normalization by document length; usually performed in similarity

Term Weighting: TF-IDF

Combine local and global importance:

$$\text{TF-IDF}(t, d) = TF(t, d) \cdot IDF(t)$$

- High weight if:
  - Term is frequent in the document.
  - Term is rare in the corpus.
- Reduces impact of common words.
- Highlights discriminative terms.
- Now terms contribute differently to the similarity computation based on their importance for the document and the corpus.

From Keywords to Weighted Vector Representations

With TF-IDF, we move from sets to weighted vectors.

- Each term represents one dimension.
- Each value is the TF-IDF weight of that term for the item.
- $x_i \in \mathbb{R}^d$, where d is the size of the vocabulary (number of unique terms across all documents).
- Items are now represented as points in a high-dimensional space.

Example (simplified vocabulary):
$V = [detective,\ murder,\ new\ york]$
$x_{\text{item}} = [0.8,\ 0.1,\ 0.5]$

Similarity: Cosine Similarity

- With the move to TF-IDF, we need to adapt our similarity measure to account for the weighted vector representations.
- Cosine Similarity measures similarity between two vectors and is the standard similarity metric for TF-IDF vectors.

$$\cos(u, x_i) = \frac{u \cdot x_i}{\|u\| \|x_i\|}$$

- Cosine similarity measures angle between vectors ➜ independent of the magnitude of the vectors.
- Values are in $[0, 1]$ for non-negative vectors.
- The higher the value, the more similar the vectors.

Content-Based Recommendation Pipeline Revisited

- Same pipeline as before, but now with improved item representation (TF-IDF vectors) and similarity measure (cosine similarity).
- The better the representation quality, the better the recommendation quality.

User Profile Representation

- User profile should be represented in the same feature space as items (e.g., TF-IDF vectors) to allow for similarity computation.
- Given item vectors $x_i \in \mathbb{R}^d$, we construct:

$$u \in \mathbb{R}^d$$

- A user is represented by the items they interacted with (e.g., liked, rated positively).
- Profile aggregates information from these items.
- User profile is a summary of the user's preferences based on past interactions with items.

User Profile Representation: Averaging

$$u = \frac{1}{|I_u|} \sum_{i \in I_u} x_i$$

where I_u is the set of items user interacted with and x_i is the item feature vector.

- Each item contributes equally.
- Captures the “center” of user preferences.
- Simple and efficient, but may not capture nuances of user preferences (e.g., if user has diverse interests, averaging may dilute them).
- Widely used baseline for content-based filtering.

User Profile Representation: Averaging cont.

Model	User Features	Track Feat.
Content avg	user AF avg	AF
Content avg, sd	user AF avg and SD	AF
Content binary k-means	avg. AF of single cluster	AF
Content weighted k-means	weighted avg. AF of clusters	AF
GMM	avg. densities of user's tracks	GMM densities
Content binary GMM	avg. AF of single GMM comp.	AF
Content weighted GMM	weighted avg. AF of GMM comp.	AF
GMM + Content avg, sd	GMM and user AF avg and SD	GMM, AF

Table 2. Overview of evaluated models (AF stands for acoustic features, GMM for Gaussian Mixture Model and SD for the standard deviation).

Model	Prec	Rec	F_1
GMM + Content avg, sd	**0.771**	**0.427**	**0.632**
Content k-means weighted	0.606	0.316	0.400
Content k-means binary	0.573	0.300	0.383
Content binary GMM	0.569	0.298	0.381
Content weighted GMM	0.569	0.298	0.381
GMM	0.231	0.122	0.226
Content avg, sd	0.161	0.089	0.241
Content avg	0.159	0.087	0.241

Table 3. Precision, Recall and F_1@10, ordered by F_1.

Zangerle, Eva, and Martin Pichl. "Content-based user models: modeling the many faces of musical preference." In 19th International Society for Music Information Retrieval conference (ISMIR). 2018.

User Profile Representation: Weighted Averaging

$$u = \frac{1}{Z} \sum_{i \in I_u} w_i \cdot x_i$$

- Weights w_i can reflect:
  - Ratings (e.g., 5 stars > 3 stars)
  - Interaction strength (purchase > click)
  - Temporal dynamics and recency (recent > old)
- Emphasizes more relevant items.
- Reduces noise from weak signals.
- More realistic representation of user preferences.

User Profile Representation: Negative Feedback

- Not all signals are positive.
- We can also incorporate negative feedback (e.g., items the user disliked, rated negatively, etc.) into the user profile representation.
- Goal: move profile toward liked items and away from disliked items.
- Distinguish positive items I_u^+ and negative items I_u^-:

$$u = \sum_{i \in I_u^+} x_i \; - \; \sum_{j \in I_u^-} x_j$$

User Profile Representation: Example

Given vocabulary $V = [detective,\ murder,\ new\ york]$ and item vectors (TF-IDF):

$$x_1 = [0.9,\ 0.1,\ 0.0] \quad \text{(detective-heavy)}$$

$$x_2 = [0.1,\ 0.8,\ 0.0] \quad \text{(murder-heavy)}$$

$$x_3 = [0.0,\ 0.1,\ 0.9] \quad \text{(new york-heavy)}$$

User interacted with items x_1 and x_2:

$$u = \frac{1}{2}(x_1 + x_2) = \frac{1}{2}([0.9, 0.1, 0] + [0.1, 0.8, 0])$$

$$u = [0.5,\ 0.45,\ 0]$$

User Profile Representation: Example cont.

$$u = [0.5,\ 0.45,\ 0]$$

Candidate items:

$$x_4 = [0.6,\ 0.5,\ 0.0]$$

$$x_3 = [0.0,\ 0.1,\ 0.9]$$

Comparison

- x_4 aligns with user interests; strong on “detective” and “murder”
- x_3 mismatches; strong on “new york” (user not interested)
- x_4 should be ranked higher than x_3

User Profile Representation: Example cont.

Now let's quantify the similarity by computing dot products:

$$u \cdot x = \sum_k u_k \cdot x_k$$

Dot product measures alignment in features (high if both vectors have large values in same dimensions).

$$u \cdot x_4 = 0.5 \cdot 0.6 + 0.45 \cdot 0.5 = 0.3 + 0.225 = 0.525$$

$$u \cdot x_3 = 0.5 \cdot 0.0 + 0.45 \cdot 0.1 = 0.045$$

User Profile Representation: Example cont.

Next, we can compute the cosine similarity to account for vector magnitudes and get a normalized similarity score between 0 and 1.

$$\cos(u, x) = \frac{u \cdot x}{\|u\| \|x\|}$$

Cosine similarity measures the angle between vectors, giving us a normalized measure of similarity that is independent of vector length (scale).

Compute magnitudes (norms):

$$\|u\| = \sqrt{0.5^2 + 0.45^2} = \sqrt{0.25 + 0.2025} = \sqrt{0.4525} \approx 0.673$$

$$\|x_4\| = \sqrt{0.6^2 + 0.5^2} = \sqrt{0.36 + 0.25} = \sqrt{0.61} \approx 0.781$$

$$\|x_3\| = \sqrt{0.1^2 + 0.9^2} = \sqrt{0.01 + 0.81} = \sqrt{0.82} \approx 0.906$$

User Profile Representation: Example cont.

$$\cos(u, x_4) = \frac{0.525}{0.673 \cdot 0.781} \approx \frac{0.525}{0.525} \approx 1.00$$

$$\cos(u, x_3) = \frac{0.045}{0.673 \cdot 0.906} \approx \frac{0.045}{0.610} \approx 0.074$$

Text Features: Extraction and Preprocessing

- So far, we have assumed that we have already extracted features (keywords) from the text content of items.
- However, in practice, items are often described by unstructured text (e.g., product descriptions, reviews, movie plots, etc.).
- We need to extract meaningful features from this text to represent items in the feature space.
- This involves several steps of text preprocessing and feature extraction, such as:
  - Tokenization: splitting text into individual words or tokens
  - Normalization: converting text to lowercase, removing punctuation, etc.
  - Stop-word removal: removing common words that do not carry much meaning (e.g., “the”, “is”, “and”, etc.)
  - Stemming/Lemmatization: reducing words to their base or root form (e.g., “running” ➔ “run”, “better” ➔ “good”)

Text Features: Extraction and Preprocessing cont.

- After preprocessing, we can represent the text as a vector of TF-IDF weights.
- The choice of preprocessing steps and feature extraction methods can significantly impact the quality of the item representation.
- Preprocessing can help reduce noise and decrease dimensionality.

Recap: TF-IDF

- TF-IDF is a widely used term weighting scheme in content-based filtering.
- It improves upon simple term frequency by considering the rarity of terms across the corpus, giving higher weight to informative terms and reducing the impact of common terms.
- But still…
  - No semantics (e.g., synonyms, polysemy).
  - No context (e.g., word order, phrases, negations) are captured by TF-IDF.
  - All words are treated independently, ignoring relationships between them ➔ shallow representation

Latent Representations

Latent Representations

- TF-IDF and keyword-based representations are shallow and do not capture deeper semantic relationships between items.
- Goal:
  - Learn dense, low-dimensional representations of items that capture latent factors or topics underlying the observed features.
  - Move beyond individual words to capture higher-level concepts and relationships between items.
  - Compress high-dimensional, sparse representations (e.g., TF-IDF) into dense vectors that capture the most important information (semantics).
  - Smaller set of abstract dimensions that capture the underlying structure of the data and relationships.
- Core idea: Co-occurrence patterns of words can reveal latent topics or factors that explain the observed features of items.

Latent Semantic Indexing (LSI)

- LSI is a technique that aims to reduce the dimensionality of the term-document matrix.
- Captures hidden structure in the data by identifying latent topics that explain the observed term usage patterns across documents.
- Document is described in a latent topic space, where each dimension corresponds to a latent topic that captures co-occurrence patterns of words across documents.
- Number of latent dimensions is a hyperparameter that controls the level of abstraction (e.g., 100-300 dimensions), substantially lower than the original vocabulary size (e.g., 10,000+ dimensions).
- In our example scenario, books about detectives and murder might be represented in a latent topic space where one dimension captures the “crime” topic, another dimension captures the “location” topic (e.g., “new york”), etc.

Latent Semantic Indexing (LSI) cont.

- Steps
  - Build term-document matrix.
  - Apply Singular Value Decomposition (SVD) to reduce dimensions to a substantially smaller set of latent topics.
  - Documents represented in latent space, in which similar documents are close ➜ semantics!

Latent Semantic Indexing (LSI) cont.

- Latent dimensions are hard to interpret
  - What makes those items similar has to be detected post-hoc, no clear semantics for a dimension (no label).
  - SVD is computationally intensive.
  - Items are linear combinations of latent dimensions, linear models have limited expressiveness.

Learned Representations (Embeddings)

- State-of-the-art is to learn representations of items and users directly from data.
- Representation includes semantic relationships between items and users, not just surface features.
- Word embeddings
  - Idea: Learn dense vector representations of words that capture semantic relationships based on their co-occurrence patterns in a large corpus of text.
    ➜ Words that appear in similar contexts have similar embeddings (distributional hypothesis).
  - Popular methods: Word2Vec, GloVe, FastText
  - Trained on large corpora (e.g., Wikipedia, Common Crawl) to capture rich semantic relationships between words.

Learned Representations (Embeddings) cont.

- (Word) Embeddings capture relationships
  - Similarity: “murder” ≈ “homicide”
  - Analogies: “king” − “man” + “woman” ≈ “queen”
  - We can compute these relationships on the vectors ➜ vector arithmetic captures semantic relationships.
- Representation of full documents by aggregation of word embeddings (e.g., averaging) or by learning sentence/document embeddings (e.g., Doc2Vec, BERT-based sentence embeddings).
- Analogously, more complex models (e.g., transformers) learn contextualized representations:
  - Word meaning depends on context.
  - Capture syntax and semantics jointly.

Content beyond Text

Content beyond Text

- Content-based filtering is not limited to text content.
- Particularly relevant for recommender systems, which often are in domains where items are described by rich multimedia content (e.g., music, movies, products, etc.).
- We can also use other types of content to represent items, such as:
  - Audio features (e.g., acoustic features of music, speech features of podcasts)
  - Image features (e.g., visual features, color, shape, texture)
  - Video features (e.g., multi-modal features, visual, audio, text)
  - Metadata (e.g., genre, user-created tags, categories)
- The same principles apply: we need to extract meaningful features from the content and represent items in a way that allows for similarity computation with user profiles.

Image Content

- For items with image content (e.g., products, movies, etc.), we can extract visual features to represent items in the feature space.
- Visual features can capture aspects such as color, shape, edges, texture, objects, etc.
- Methods for extracting visual features include:
  - Traditional computer vision techniques (e.g., SIFT, HOG, color histograms, etc.)
  - Deep learning-based methods (e.g., convolutional neural networks) that learn hierarchical feature representations from raw pixel data.

Audio Content

- For items with audio content (e.g., music, podcasts, etc.), we can extract acoustic features to represent items in the feature space.
- Acoustic features can capture aspects such as pitch, tempo, rhythm, timbre, etc.
- Methods for extracting acoustic features include:
  - Traditional signal processing techniques (e.g., Mel-frequency cepstral coefficients (MFCCs), chroma features, spectral contrast, etc.)
  - Deep learning-based methods that learn feature representations from raw audio data (e.g., using convolutional or recurrent neural networks).

Audio Content cont.

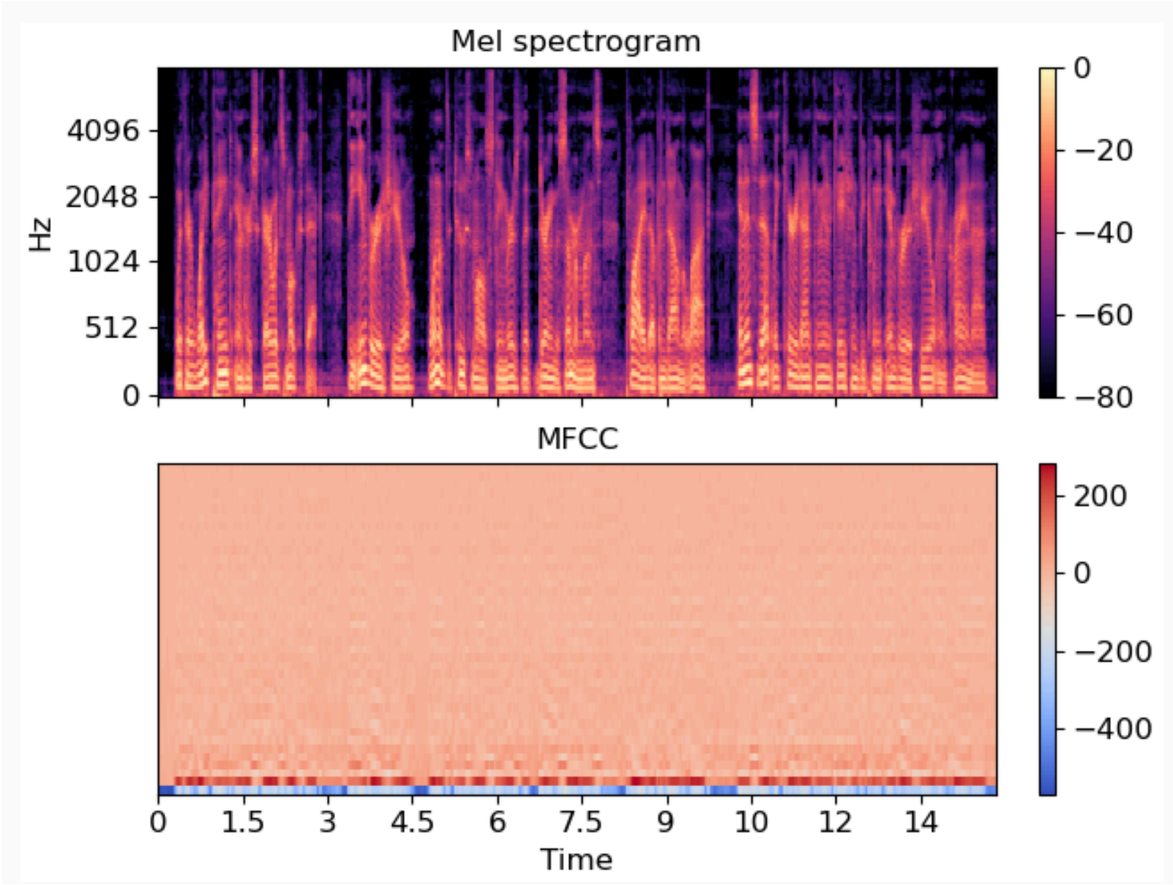


Figure taken from librosa documentation (link).

Video Content

- Videos are inherently multi-modal, capturing
  - Visual content (objects, scenes, colors)
  - Motion and temporal dynamics (actions, events)
  - Audio track (speech, music, sound effects)
- Methods for extracting video features include:
  - Frame-based visual features (e.g., CNNs applied to individual frames)
  - Spatio-temporal models capturing motion (e.g., 3D CNNs, optical flow, video transformers)

Multimodal Content

- Many items have multiple types of content (e.g., movies: text descriptions, images, audio).
- Multimodal content can provide richer information about items and allow for better recommendations.
- Extract features from modalities, combine them into a multimodal feature space.
- Methods for combining multimodal features include:
  - Early fusion: concatenate features from modalities into a single vector.
  - Late fusion: compute similarity separately for each modality and combine similarity scores.
  - Joint learning: learn a unified representation from all modalities simultaneously (e.g., deep learning models that take multiple inputs).
- Multimodal representations can capture complementary information from different content types, leading to improved recommendation performance.

Multimodal Data

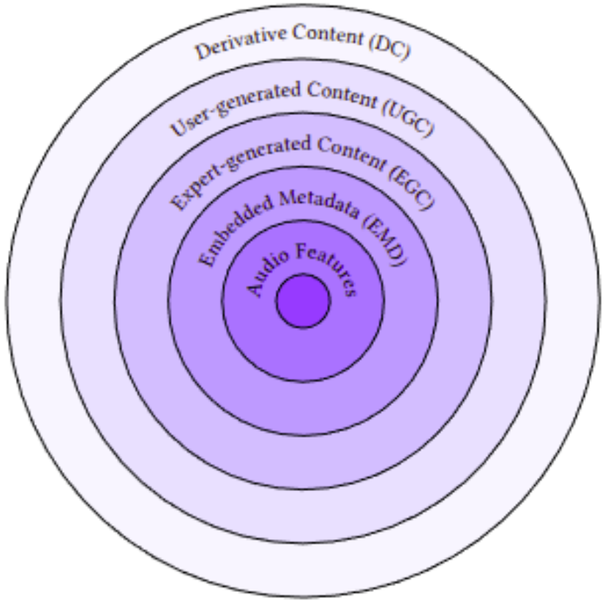


Figure 1: Onion model of music features.

Table 2: Features introduced in Music4All-Onion, categorized according to the layers of the onion model.

Layer	Features
Audio	Acoustic (Short-term, Block-level)
EMD	Lyrics (preprocessed, tf-idf, word2vec, emotions)
EGC	Genres (tf-idf)
UGC	Tags (dictionary with Last-FM API weights, tf-idf)
DC	Video (VGG19, Incp3, Resnet)

Marta Moscati, Emilia Parada-Cabaleiro, Yashar Deldjoo, Eva Zangerle, and Markus Schedl. 2022. Music4All-Onion – A Large-Scale Multi-faceted Content-Centric Music Recommendation Dataset. In Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM '22). Association for Computing Machinery, New York, NY, USA, 4339–4343. https://doi.org/10.1145/3511808.3557656

Multimodal Content cont.

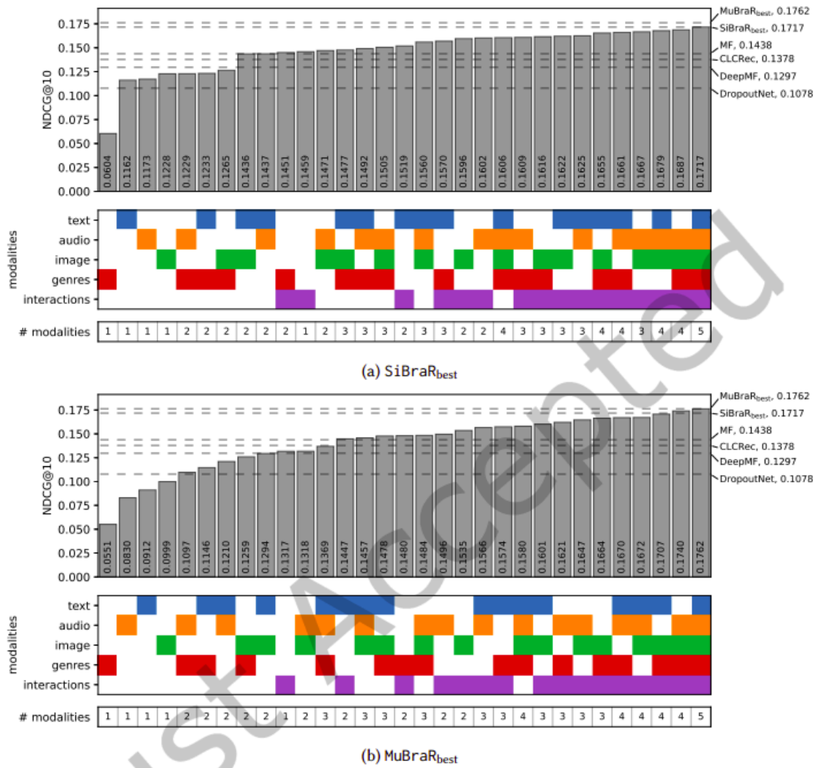


Fig. 6. NDCG@10 results of (a) SiBraR$_{best}$ and (b) MuBraR$_{best}$ trained on warm-start Onion with interactions, genres, image, audio, and text modalities. The bars correspond to evaluations on the test set with varying evaluation modalities. If a modality is used, its block is filled with the corresponding color in the central plot. The bottom plot counts the number of modalities per bar. The gray dashed horizontal lines in the bar plot show the NDCG@10 of the best CFs and HRSs in warm-start Onion.

Just Accepted

Christian Ganhör, Marta Moscati, Anna Hausberger, Shah Nawaz, and Markus Schedl. 2025. Single-Branch Network Architectures to Close the Modality Gap in Multimodal Recommendation. ACM Trans. Recomm. Syst. Just Accepted (September 2025). https://doi.org/10.1145/3769430